

SKILL WEEK-2

2 nd question

Task-1:

```
arikel@DESKTOP-AMUSJME:~$ ps -f
UID          PID    PPID  C STIME TTY          TIME CMD
arikel      350      349  0 05:19 pts/0        00:00:00 -bash
arikel      714      350  99 05:39 pts/0        00:00:00 ps -f
arikel@DESKTOP-AMUSJME:~$ echo $$
350
arikel@DESKTOP-AMUSJME:~$ pstree -p
systemd(1)─agetty(219)
           └─chronyd-starter(154)─chronyd(258)─chronyd(264)
           └─cron(155)
           └─dbus-daemon(156)
           └─init-systemd(Ub(2))─SessionLeader(348)─Relay(35+
                                   └─init(7)─{init}(8)
                                   └─login(351)─bash(434)
                                   └─{init-systemd(Ub)}(9)
           └─networkd-dispat(162)
           └─rsyslogd(187)─{rsyslogd}(216)
                           └─{rsyslogd}(217)
                           └─{rsyslogd}(218)
           └─systemd(400)─(sd-pam)(402)
           └─systemd-journal(50)
           └─systemd-logind(174)
           └─systemd-resolve(85)
           └─systemd-udevd(91)
           └─unattended-upgr(238)─{unattended-upgr}(304)
arikel@DESKTOP-AMUSJME:~$ man 2 fork
```

fork(2)

System Calls Manual

fork(2)

NAME

fork - create a child process

LIBRARY

Standard C library (libc, -lc)

SYNOPSIS

#include <unistd.h>

pid_t fork(void);

DESCRIPTION

fork() creates a new process by duplicating the calling process. The new process is referred to as the child process. The calling process is referred to as the parent process.

The child process and the parent process run in separate memory spaces. At the time of fork() both memory spaces have the same content. Memory writes, file mappings (mmap(2)), and unmappings (munmap(2)) performed by one of the processes do not affect the other.

The child process is an exact duplicate of the parent process except for the following points:

- The child has its own unique process ID, and this PID does not match the ID of any existing process group (setpgid(2)) or session.
- The child's parent process ID is the same as the parent's process ID.
- The child does not inherit its parent's memory locks

```
arikel@DESKTOP-AMUSJME: x + v - □ ×
exec(3) Library Functions Manual exec(3)

NAME
    execl, execlp, execl, execv, execvp, execvpe - execute
    a file

LIBRARY
    Standard C library (libc, -lc)

SYNOPSIS
    #include <unistd.h>

    extern char **environ;

    int execl(const char *path, const char *arg, ...
              /*, (char *) NULL */);
    int execlp(const char *file, const char *arg, ...
              /*, (char *) NULL */);
    int execl(const char *path, const char *arg, ...
              /*, (char *) NULL, char *const envp[] */);
;
    int execv(const char *path, char *const argv[]);
    int execvp(const char *file, char *const argv[]);
    int execvpe(const char *file, char *const argv[], char *c
onst envp[]);

    Feature Test Macro Requirements for glibc (see fea-
    ture_test_macros(7)):

    execvpe():
        _GNU_SOURCE

DESCRIPTION
    The exec() family of functions replaces the current
    process image with a new process image. The functions
    described in this manual page are layered on top of ex-
    Manual page exec(3) line 1 (press h for help or q to quit)
```

```
arikel@DESKTOP-AMUSJME:~$ man 2 fork
arikel@DESKTOP-AMUSJME:~$ man 3 exec
arikel@DESKTOP-AMUSJME:~$ sleep 30 &
[1] 875
arikel@DESKTOP-AMUSJME:~$ ps -f
UID          PID     PPID  C  STIME TTY          TIME CMD
arikel      350       349  0  05:19 pts/0        00:00:00 -bash
arikel      875       350  0  05:52 pts/0        00:00:00 sleep 30
arikel      885       350  0  05:52 pts/0        00:00:00 ps -f
```

```

arikel@DESKTOP-AMUSJME:~$ pstree -p
systemd(1)─agetty(219)
          └─chronyd-starter(154)─chronyd(258)─chronyd(264)
          └─cron(155)
          └─dbus-daemon(156)
          └─init-systemd(Ub(2)─SessionLeader(348)─Relay(35+
                                └─init(7)─{init}(8)
                                └─login(351)─bash(434)
                                └─{init-systemd(Ub}(9)
          └─networkd-dispat(162)
          └─rsyslogd(187)─{rsyslogd}(216)
                        └─{rsyslogd}(217)
                        └─{rsyslogd}(218)
          └─systemd(400)─(sd-pam)(402)
          └─systemd-journal(50)
          └─systemd-logind(174)
          └─systemd-resolve(85)
          └─systemd-udevd(91)
          └─unattended-upgr(238)─{unattended-upgr}(304)
          └─wsl-pro-service(831)─{wsl-pro-service}(832)
                                └─{wsl-pro-service}(833)
                                └─{wsl-pro-service}(834)
                                └─{wsl-pro-service}(835)
                                └─{wsl-pro-service}(836)
                                └─{wsl-pro-service}(837)
                                └─{wsl-pro-service}(840)

arikel@DESKTOP-AMUSJME:~$ bash -c 'echo Before;exec ls;echo aft
er'
Before
OSSP  OS_LAB  filename  filename.c  namedirectory

```

Task-2:

```
GNU nano 8.7.1 task2.c
#include <stdio.h>
#include <unistd.h>
int main(){
pid_t pid;
pid=fork();

if(pid<0){
printf("Fork failed!\n");
}
else if(pid==0){
printf("Child Process\n");
printf("Child PID= %d\n",getpid());
}
else{
printf("Parent Process\n");
printf("Parent PID =%d\n",getpid());
printf("Child PID =%d\n",pid);
}
return 0;
}

[ Read 20 lines ]
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify
```

```
arikela@DESKTOP-AMUSJME:~$ gcc task2.c -o task2
arikela@DESKTOP-AMUSJME:~$ ./task2
Parent Process
Parent PID =1425
Child PID =1426
Child Process
Child PID= 1426
```

```
arikel@DESKTOP-AMUSJME: x + v - □ ×
GNU nano 8.7.1 ecec_demo.c
#include <stdio.h>
#include <unistd.h>
int main(){
printf("Before exec\n");
char *args[]={"ls",NULL};
execvp(args[0],args);
printf("After ecec\n");
return 0;
}|

[ Wrote 9 lines ]
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify
```

```
arikel@DESKTOP-AMUSJME:~$ gcc exec_demo.c -o exec_demo
arikel@DESKTOP-AMUSJME:~$ ./exec_demo
Before exec
OSSP      ecec_demo.c  filename      task2
OS_LAB    exec_demo    filename.c    task2.c
ecec_demo exec_demo.c  namedirectory
arikel@DESKTOP-AMUSJME:~$
```

```
GNU nano 8.7.1 fork_exec.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(){
pid_t pid;
pid=fork();

if(pid<0){
printf("Fork Failed!\n");
}
else if(pid==0){
printf("Child Process\n");
char *args[]={"ls",NULL};
execvp(args[0],args);
//executes if execp fails
printf("exec failed!\n");
}
else{
printf("Parent Process\n");
wait(NULL);
printf("Child finished\n");
}
return 0;
}

[ Read 25 lines ]
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify
```

```
arikel@DESKTOP-AMUSJME:~$ nano fork_exec.c
arikel@DESKTOP-AMUSJME:~$ gcc fork_exec.c -o fork_exec
arikel@DESKTOP-AMUSJME:~$ ./fork_exec
Parent Process
Child Process
OSSP      ecec_demo.c  filename    fork_exec.c  task2.c
OS_LAB    ecec_demo   filename.c  namedirectory
ecec_demo ecec_demo.c fork_exec   task2
Child finished
```

GNU nano 8.7.1

task2.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main(){
char command[100];
printf("Enter command: ");
scanf("%s",command);
pid_t pid=fork();

if(pid<0){
printf("Fork failed!\n");
}
else if(pid==0){
printf("Child Process\n");
char *args[]={command,NULL};
execvp(args[0],args);
printf("Command execution failed!\n");
}
else{
printf("Parent Process\n");
printf("Parent PID =%d\n",getpid());
printf("Child PID =%d\n",pid);
wait(NULL);
printf("Child process completed.\n");
}
return 0;
}
```

[Read 27 lines]

^G Help

^O Write Out

^F Where Is

^K Cut

^T Execute

^X Exit

^R Read File

^_ Replace

^U Paste

^J Justify


```
arikel@DESKTOP-AMUSJME:~$ nano task2.c
arikel@DESKTOP-AMUSJME:~$ gcc task2.c -o task2
arikel@DESKTOP-AMUSJME:~$ ./task2
Enter command: ls
Parent Process
Parent PID =2200
Child PID =2203
Child Process
OSSP      ecec_demo.c  filename  fork_exec.c  task2.c
OS_LAB    exec_demo   filename.c  namedirectory
ecec_demo exec_demo.c  fork_exec  task2
Child process completed.
```